

High-Concurrency Distributed Ledger Service

Anarghya Das

Requirements, Software System

Indian Institute of Technology Jodhpur

Jodhpur, India

b24cs1096@iitj.ac.in

Arnav Vivek

Requirements, Team

Indian Institute of Technology Jodhpur

Jodhpur, India

b24cs1014@iitj.ac.in

Neeli Satwik

Requirements, Work Opportunity

Indian Institute of Technology Jodhpur

Jodhpur, India

b24cs1048@iitj.ac.in

Subhanshu Gupta

Requirements, Stakeholders,

Way of Working

Indian Institute of Technology Jodhpur

Jodhpur, India

b24cs1072@iitj.ac.in

I. PROBLEM STATEMENT

A. Background

The rapid growth of digital payment platforms, online banking systems, and financial technology applications has significantly increased the demand for high-throughput transaction processing systems. These systems must handle large volumes of concurrent monetary transfers while guaranteeing strong correctness properties such as atomicity, consistency, isolation, and durability. Even a single inconsistency, such as a partial transfer or an incorrect balance update, can lead to financial loss and loss of trust.

Traditional centralized database systems provide strong ACID guarantees but suffer from limited scalability. As the number of concurrent users increases, vertical scaling becomes insufficient due to hardware limits on CPU, memory, disk throughput, and network bandwidth. Consequently, modern transaction systems adopt distributed architectures in which data is partitioned across multiple nodes to enable horizontal scaling. However, distributing data introduces significant complexity in maintaining consistency across shards, especially when transactions span multiple partitions.

B. Core Challenges

Designing a distributed ledger service that scales horizontally while preserving strong transactional guarantees presents several fundamental challenges:

- 1) **Atomic Cross-Shard Execution:** When a transaction involves accounts stored on different shards, the system must ensure that either all updates commit or all abort. Partial commits must never occur.
- 2) **Durability Under Failures:** The system must guarantee that once a transaction is committed, it survives process crashes, replica failures, and leader changes.
- 3) **Consistency Preservation:** Financial invariants must be maintained at all times, including:
 - Conservation of total monetary value across the system.

- Prevention of negative account balances.
- No duplication or loss of transactions.

- 4) **High Concurrency:** The system must process large numbers of concurrent transactions without introducing race conditions or violating isolation properties.
- 5) **Fault Tolerance:** The system must tolerate single-node failures without data loss and must recover automatically through the help of replicas and log replay using WAL.
- 6) **Scalability Without Global Coordination Bottlenecks:** Adding shards should increase throughput proportionally, without introducing centralized coordination limits.

C. Problem Definition

The problem addressed in this work is the design and implementation of a horizontally scalable distributed ledger service that supports real-time monetary transfers with strong consistency guarantees under concurrent and failure-prone environments.

Formally, the system must satisfy the following requirements:

- Accept transactions consisting of a unique identifier, source account, destination account, and transfer amount.
- Execute single-shard transactions with low latency and minimal coordination overhead.
- Execute cross-shard transactions atomically using two phase commit protocols.
- Persist all write operations using a write-ahead logging mechanism prior to state modification.
- Replicate shard data across multiple replicas and commit writes only after majority acknowledgment.
- Automatically detect and recover from database failures.
- Preserve correctness invariants even during crash recovery and reprocessing.

The system must operate on commodity hardware within a single datacenter environment, assume fail-stop node failures, and rely on crash-recovery storage semantics. Byzantine

behavior and geo-distributed consistency guarantees are explicitly out of scope.

D. Objectives and Target Metrics

Beyond functional correctness, the system aims to achieve the following measurable performance objectives:

- Support a minimum throughput of 100 transactions per second per shard.
- Maintain single-shard transaction latency below 50 ms at the 99th percentile.
- Maintain cross-shard transaction latency below 200 ms at the 99th percentile.
- Scale throughput horizontally with the addition of new shards.

These targets reflect a balance between strong consistency guarantees and practical performance expectations for a datacenter-scoped distributed ledger.

E. Motivation

While large-scale distributed databases such as Google Spanner and CockroachDB provide sophisticated global consistency mechanisms, their architectures rely on complex consensus protocols, tightly synchronized clocks, and extensive infrastructure. There is significant educational value in designing a simplified yet realistic distributed ledger that exposes the underlying principles of sharding, distributed coordination, replication, logging, and failure recovery.

This project therefore seeks to construct a high-concurrency ledger service that captures the essential mechanisms of modern distributed transaction engines while maintaining architectural clarity and traceability from requirements to implementation.

II. REVIEW OF EXISTING TECHNOLOGIES AND SOLUTIONS

Designing a high-concurrency distributed ledger requires balancing scalability, consistency, durability, and operational simplicity. Modern distributed databases address these concerns through sharding, replication, consensus protocols, and multi-version concurrency control. This section evaluates two prominent distributed database systems, CockroachDB and FoundationDB, and analyzes their architectural strengths and trade-offs in the context of a high-concurrency ledger service.

A. FoundationDB

FoundationDB is a distributed transactional key-value store that provides strict ACID guarantees and automatic data distribution. Its architecture separates transaction processing from storage, internally handling sharding, replication, logging, and recovery.

1) Strengths: Built-in Sharding, Two-Phase Commit, WAL, and Replication: FoundationDB automatically performs key-range sharding, distributed transaction coordination, write-ahead logging, and consensus-based replication. These mechanisms are transparent to application developers, significantly reducing implementation complexity.

High Concurrency Performance: The system employs Multi-Version Concurrency Control (MVCC) combined with optimistic concurrency control. Non-conflicting transactions execute in parallel while preserving serializable isolation. This design enables high throughput in workloads where contention is moderate.

Automatic Hot Range Migration: FoundationDB continuously monitors key-range load. Overloaded ranges are automatically split and redistributed across storage servers without application-level intervention. This dynamic rebalancing enhances throughput scalability and reduces operational overhead.

2) Limitation: Limited Visibility into Recovery Internals: Core mechanisms such as WAL management, replication logic, and failover orchestration are abstracted away. While beneficial for operational simplicity, this abstraction reduces transparency and fine-grained control, which may limit experimentation and educational exploration of recovery semantics.

B. CockroachDB

CockroachDB is a distributed SQL database that provides PostgreSQL compatibility over a shared-nothing transactional key-value store. It uses the Raft consensus algorithm for replication and Multi-Version Concurrency Control to ensure serializable isolation.

1) Primary Strengths: Strong ACID Guarantees with Horizontal Scalability: By combining Raft-based consensus replication with MVCC, CockroachDB ensures strict consistency across distributed nodes. Data is partitioned into ranges, each managed by a Raft group, enabling linear scaling of read and write throughput by adding nodes.

High Availability and Fault Tolerance: Every write operation is replicated to a quorum of nodes before commit, ensuring durability even under node failures. Automatic range rebalancing and replica management provide near-zero Recovery Point Objectives and Recovery Time Objectives.

Global Multi-Region Deployment: The system supports geo-partitioning and row-level data placement, allowing latency optimization and regulatory compliance across regions.

2) Architectural Trade-offs: Consensus Latency Overhead: Each write operation requires coordination with a Raft quorum. This introduces additional network round trips and increases commit latency compared to single-node databases. For low-latency financial transfers, this overhead may be non-negligible.

Resource Intensity: Background processes such as log compaction, range rebalancing, and consensus maintenance require substantial CPU and memory resources. Basic fault tolerance requires a minimum of three nodes, increasing infrastructure cost and complexity.

Clock Synchronization Sensitivity: The system depends on reasonably synchronized clocks across nodes. Significant clock skew may degrade performance or cause node instability. While manageable in production environments, this introduces operational considerations.

OLAP Limitations: CockroachDB is optimized for transactional workloads. Large analytical queries involving massive scans or complex joins are less efficient compared to columnar database systems.

C. Comparative Analysis in the Context of a High-Concurrency Ledger

Both FoundationDB and CockroachDB provide mature, production-grade solutions for distributed transactions. However, when evaluated specifically for a high-concurrency ledger service designed for educational exploration and architectural transparency, certain trade-offs emerge:

- FoundationDB abstracts core mechanisms such as logging and failover, limiting visibility into internal durability and recovery processes.
- CockroachDB introduces consensus-induced latency overhead on every write, which may increase commit times for simple monetary transfers.
- Operational complexity and resource requirements are significantly higher than a purpose-built, datacenter-scoped ledger engine.

These observations motivate the development of a custom distributed ledger architecture that retains core distributed systems principles such as sharding, write-ahead logging, majority-based replication, and two-phase commit coordination, while maintaining architectural clarity and measurable performance characteristics. The proposed system prioritizes deterministic partitioning, stateless coordination, and explicit failure recovery mechanisms, enabling fine-grained control and experimental evaluation of distributed transaction behavior.

III. SOLUTION TRAJECTORY

The High-Concurrency Ledger Service is designed as a sharded, log-driven distributed system deployed within a single datacenter environment. The architecture emphasizes separation of concerns between transaction ingestion, coordination, storage, replication, and operational monitoring. This layered structure enables horizontal scalability while preserving strict financial correctness guarantees.

A. Requirements Engineering

Our development followed an iterative requirements engineering approach, resulting in seven major revisions of the SRS document over a two-week period (January 20 to February 4, 2026). Each iteration addressed specific stakeholder feedback, technical constraints, or design insights.

IV. SYSTEM ARCHITECTURE

The system is composed of the following primary components:

- *API Gateway*
- *Event Streaming Layer (Kafka)*
- *Transaction Coordinator*
- *Shard Map Service*
- *Shards (Leaders and Followers)*

TABLE I
SRS VERSION EVOLUTION

Ver	Date	Key Changes
1.0	Jan 20	Initial SRS document
1.1	Jan 21	Added REQ-TX-002 and REQ-TX-003; updated architecture to follow sharding pattern
1.2	Jan 24	Added REQ-REP-003 and REQ-REP-004 (failure detection and primary failover)
1.3	Jan 26	Added write-ahead logging requirements (REQ-DATA-002); documented WAL recovery semantics
2.0	Jan 28	Added Kafka integration (REQ-COORD-001); defined stateless coordinator model
2.1	Jan 30	Added load balancing requirements (REQ-DATA-003); documented shard rebalancing strategy
2.2	Jan 31	Added performance requirements (REQ-PERF-001 to REQ-PERF-004); defined latency targets
2.3	Feb 02	Major revision: Tentative system architecture; enhanced UML diagrams; updated sharding techniques; documented naming convention; clarified dataflow and system interactions; added traceability matrix

• *Shard Load Monitor*

These components interact to form a write-optimized transactional pipeline that decouples client interaction from data persistence and execution.

A. API Gateway

The API Gateway serves as the entry point for client requests. Its responsibilities include:

- Validation of transaction structure and parameters.
- Enforcement of rate limiting and request sanitization.
- The API Gateway acts as a Kafka producer, publishing validated transaction events to a streaming topic for downstream coordination.

B. Event Streaming Layer

All validated transactions are published by the API Gateway to the event streaming layer before execution. This layer serves as the durable ingestion backbone of the system and provides:

- Persistent storage of transaction events.
- Ordered delivery within partitions.
- Replay capability for failure recovery.

Transaction Coordinators operate as stateless consumers within a consumer group. Each coordinator processes assigned partitions and commits offsets only after successful handling of events.

In the event of coordinator failure, partition ownership is reassigned to another coordinator instance, which resumes consumption from the last committed offset. Because shard-level execution is idempotent and durable state changes are

recorded in shard-level write-ahead logs, replayed events do not result in duplicate or inconsistent state transitions.

C. Coordination Layer

The coordination layer consists of the **Transaction Coordinator** and the **Shard Map**.

1) *Transaction Coordinator*: The Transaction Coordinator is responsible for:

- Consuming transaction events from the distributed log.
- Determining the target shard(s) for involved accounts.
- Routing requests to appropriate shard leaders.
- Orchestrating two-phase commit when multiple shards are involved.

The Transaction Coordinator maintains no persistent transaction metadata locally. All durability and recovery guarantees are delegated to the distributed ingestion log and shard-level write-ahead logging mechanisms. As a result, coordinators can fail and restart without requiring state reconstruction, since transaction progress can be safely resumed through log replay and idempotent shard execution.

2) *Shard Map*: The Shard Map maintains the mapping between logical partitions and physical shard servers. Account identifiers are deterministically mapped to partitions using a hash-based scheme. Logical partitions may be reassigned to different physical shards to support load balancing. Additionally, the shard map will be updated during rebalancing.

This indirection layer enables horizontal scalability while maintaining consistent routing.

D. Data Layer: Shards and Replicas

The data layer is composed of multiple shards, each managing a subset of account data. Data within each shard is further divided into smaller logical partitions. For example, a shard may store up to ten partitions, where each partition corresponds to a range of hash values. The shard map maintained by the coordinator tracks partition placement and is updated following rebalancing operations.

Each shard follows a primary–follower architecture:

- A single leader handles all write operations.
- Followers replicate state from the leader.
- Majority-based acknowledgment ensures consistency and fault tolerance.

Shards are responsible for validating transactions, maintaining account balances, and enforcing ledger invariants such as non-negative balances and conservation of total funds.

E. Operational Monitoring Layer

The Shard Load Monitor continuously observes shard health and workload metrics, including request rate and system utilization. When sustained imbalances are detected, controlled partition migration may be initiated on a weekly or bi-weekly basis.

Partition migration is coordinated to ensure transactional consistency. Logical partitions are temporarily halted during transfer, remapped upon successful migration, and then resumed.

Algorithm 1 Single-Shard Transaction Execution

Require: $txnID$, $source$, $destination$, $amount$

Ensure: Atomic and durable ledger update

```

1: if  $balance(source) < amount$  then
2:   return REJECT
3: end if
4: Append  $(txnID, operation, UNCOMMITTED)$  to WAL
5: Replicate WAL entry to followers
6: Wait for majority ACK
7: Advance commit index
8: Apply debit and credit to ledger state
9: return SUCCESS

```

V. TRANSACTION PROCESSING MODEL

The ledger supports both single-shard and cross-shard transactions. The execution model is designed to preserve strict financial invariants while minimizing coordination overhead wherever possible. Each transaction is defined as

$$T = (txnID, sourceAccount, destinationAccount, amount) \quad (1)$$

The system enforces the following invariants at all times:

- **No negative account balances:**

$$balance(account) \geq 0$$

- **Conservation of total monetary value:**

$$\sum balance_{before} = \sum balance_{after}$$

- **No partial commits.**

Transactions are first ingested through the messaging layer and routed by the Transaction Coordinator to the appropriate shard(s). Execution behavior depends on whether the transaction spans one shard or multiple shards.

A. Single-Shard Transaction Execution

A transaction is classified as single-shard if both accounts map to the same shard according to deterministic partitioning.

In this case, distributed coordination is unnecessary, and execution is handled entirely by the shard leader. The procedure is shown in Algorithm 1.

Execution Semantics: If validation succeeds, the shard appends an uncommitted record to its Write-Ahead Log (WAL). The log entry is replicated to follower replicas and must be acknowledged by a majority before commit index advancement.

State changes are applied strictly after durable replication. This ensures that:

- A committed transaction survives shard crashes.
- No state modification occurs without durable logging.

Because the shard leader serializes conflicting operations on the same account, isolation is naturally enforced at shard scope.

Algorithm 2 Cross-Shard Coordination (Two-Phase Commit)

Require: $txnID$, participating shards S

```
1: for all  $S_i \in S$  do
2:   Send PREPARE to  $S_i$ 
3: end for
4: if all shards respond PREPARED then
5:   for all  $S_i \in S$  do
6:     Send COMMIT to  $S_i$ 
7:   end for
8:   return SUCCESS
9: else
10:  for all  $S_i$  that responded PREPARED do
11:    Send ABORT to  $S_i$ 
12:  end for
13:  return ABORT
14: end if
```

Algorithm 3 Shard Prepare Handling

Require: PREPARE($txnID$, operation)

```
1: if validation fails then
2:   return ABORT
3: end if
4: Append ( $txnID$ , PREPARED) to WAL
5: Replicate to followers
6: Wait for majority ACK
7: return PREPARED
```

B. Cross-Shard Transaction Execution

When source and destination accounts reside on different shards, atomicity must be enforced across independent storage units. The system employs a two-phase commit (2PC) protocol coordinated by the Transaction Coordinator.

Let $S = \{S_1, S_2, \dots, S_n\}$ denote the participating shards.

1) *Coordinator Protocol*: The coordinator executes the procedure described in Algorithm 2.

2) *Shard-Side Prepare Handling*: Each shard processes PREPARE messages as shown in Algorithm 3.

Commit Phase: Upon receiving a COMMIT message, each shard:

- Appends a COMMITTED record to its WAL.
- Replicates the record to followers.
- Advances its commit index.
- Applies the ledger update.

If ABORT is received, the shard appends an ABORTED record and releases any reserved state.

C. Atomicity Guarantee

Atomicity across shards is achieved because:

- No shard applies state updates during the PREPARE phase.
- All shards must durably log PREPARED before a COMMIT decision.
- If any shard aborts, all prepared shards receive ABORT.

Thus, the transaction either commits everywhere or nowhere.

D. Idempotency and Log Replay

Transaction Coordinators are stateless consumers of the distributed log. In failure scenarios, messages may be replayed. To prevent duplicate execution:

- Each shard tracks transaction identifiers.
- Duplicate PREPARE or COMMIT requests are detected and ignored.
- State transitions are enforced deterministically via WAL records.

This ensures correctness under coordinator failure and message replay.

E. Performance Implications

Single-shard transactions require:

- One WAL append.
- One replication round.

Cross-shard transactions require:

- Two WAL appends per shard (PREPARE and COMMIT).
- Two replication rounds.
- Additional coordinator messaging overhead.

Therefore, cross-shard transactions incur higher latency due to additional coordination and durability guarantees.

F. Design Trade-offs

The transaction processing model prioritizes correctness over minimal latency:

- WAL-before-apply semantics increase durability.
- Majority-based replication increases fault tolerance.
- Two-phase commit introduces coordination overhead but guarantees atomicity.

This structured execution model ensures financial safety while maintaining horizontal scalability across shards.

VI. REPLICATION AND FAILOVER MECHANISM

To ensure durability and fault tolerance, each shard follows a primary–follower replication architecture. Replication guarantees that committed transactions survive node failures and that the system maintains ledger invariants under crash scenarios.

A. Primary–Follower Architecture

Each shard consists of:

- One **Primary (Leader)** responsible for handling all write operations.
- Multiple **Followers** that replicate the primary’s write-ahead log (WAL).

All client-visible commits originate from the primary. Followers do not serve writes but maintain synchronized replicas of shard state.

This architecture isolates write serialization to a single node per shard, simplifying conflict resolution while enabling horizontal scalability across shards.

B. Quorum-Based Commit Rule

The system employs a majority-based acknowledgment policy. A write is considered committed only after:

- 1) The primary appends the WAL entry locally.
- 2) The WAL entry is replicated to follower replicas.
- 3) A majority of replicas acknowledge persistence.

Let N denote the total number of replicas in a shard and $Q = \lfloor N/2 \rfloor + 1$ denote the quorum size.

A transaction is committed only if at least Q replicas persist the corresponding WAL entry.

Safety Intuition: Because any two majorities intersect, at least one replica in any future quorum will contain the committed entry. Therefore:

- Committed transactions cannot be lost.
- A newly elected leader will always contain all previously committed entries.

This intersection property forms the basis of durability under leader failure.

C. Replication Procedure

For each write request, the primary executes:

- 1) Append entry to local WAL.
- 2) Send WAL entry to followers.
- 3) Followers persist entry to stable storage.
- 4) Followers acknowledge persistence.
- 5) Primary advances commit index upon quorum.
- 6) Committed entry is applied to ledger state.

State application occurs strictly after quorum confirmation to ensure crash safety.

D. Shard Leader State Machine

The Shard Leader operates as a deterministic replicated state machine. Its responsibility is to transition the shard from one consistent state to another by processing ordered transaction commands while enforcing durability, majority-based replication, and atomic application semantics.

State Definition: The shard state consists of:

- Ledger balances
- Write-Ahead Log (WAL)
- Commit index
- Last applied index
- Prepared transaction records (for cross-shard 2PC)

Formally, the shard state may be represented as:

$$State = \{ledgerBalances, walLog, commitIndex, lastApplied, preparedTxns\}$$

Single-Shard State Transitions: For a single-shard transaction, the leader performs the following transitions:

- 1) **VALIDATE:** Check balance and invariants.
- 2) **WAL_APPEND:** Append uncommitted entry to WAL and fsync.
- 3) **REPLICATE:** Send entry to followers.
- 4) **QUORUM_ACK:** Wait for majority acknowledgment.
- 5) **COMMIT:** Advance commit index.

- 6) **APPLY:** Apply debit/credit to ledger state.

State changes are applied strictly after quorum confirmation, ensuring that no ledger update occurs without durable replication.

Cross-Shard Prepare/Commit Transitions: During two-phase commit (2PC):

- **PREPARE:** Append `PREPARED` record to WAL, replicate, wait for quorum.
- **COMMIT:** Append `COMMITTED` record, replicate, advance commit index, apply.
- **ABORT:** Append `ABORTED` record and release reserved state.

No state modification occurs during `PREPARE`, guaranteeing atomicity across shards.

Crash Recovery Transitions: Upon restart, the shard replays its WAL sequentially:

- Apply entries marked `COMMITTED`.
- Retain `PREPARED` entries for coordinator reconciliation.
- Discard `UNCOMMITTED` entries.

This ensures that the recovered ledger state is equivalent to the pre-crash committed state.

Safety Guarantee: The shard leader's state machine enforces the strict ordering:

$$Persist \rightarrow Replicate \rightarrow Quorum \rightarrow Commit \rightarrow Apply$$

This ordering guarantees:

- No partial state application
- No loss of committed transactions
- Preservation of financial invariants under crash and failover

E. Failure Detection

Shard nodes are monitored using periodic heartbeats. The failure detection mechanism operates as follows:

- Each replica sends heartbeat messages at fixed intervals.
- If a replica misses multiple consecutive heartbeats, it is marked as failed.
- The Replica Manager initiates leader re-election when the primary is declared failed.

The system assumes a fail-stop model in which failed nodes cease operation rather than behaving arbitrarily.

F. Leader Failover Procedure

When the primary fails, the system performs controlled promotion of a follower.

Algorithm 4: Leader Failover

Promotion Criteria: The follower with the most up-to-date WAL (highest replicated log index) is selected to minimize rollback risk.

Recovery Semantics: Upon promotion:

- The new leader replays committed WAL entries.
- Uncommitted entries are discarded or rolled back.
- The shard resumes normal operation.

Because committed entries exist on a quorum, the promoted leader necessarily contains all committed state.

Algorithm 4 Leader Promotion Procedure

Require: Set of followers F

- 1: Identify follower with highest log index
 - 2: Promote selected follower to Primary
 - 3: Replay WAL entries to reconstruct in-memory state
 - 4: Update shard registry with new primary
 - 5: Resume write processing
-

G. Failure Scenarios and Safety Analysis

1) *Primary Crash Before Quorum*: If the primary crashes before receiving majority acknowledgment:

- The entry is not committed.
- Followers may contain partial logs.
- The new leader discards uncommitted entries.

No state modification was applied; therefore, invariants remain preserved.

2) *Primary Crash After Quorum*: If the primary crashes after quorum acknowledgment but before responding to the coordinator:

- The entry is already committed on a majority.
- The promoted leader contains the committed entry.
- The transaction remains durable.

Coordinator retries are handled via idempotency.

3) *Follower Failure*: If a follower fails:

- Writes continue as long as quorum is maintained.
- The failed replica can later rejoin and catch up via WAL synchronization.

H. Consistency Guarantees

The replication model ensures:

- **Durability**: Committed entries survive primary failure.
- **Linearizable Writes at Shard Scope**: Writes are serialized through the leader.
- **No Split-Brain**: Majority requirement prevents divergent leaders.

Combined with the transaction processing model, replication guarantees that:

- No committed monetary transfer is lost.
- No duplicate transfers are applied.
- Ledger invariants are preserved under crash and failover.

I. Design Trade-offs

The primary–follower approach simplifies implementation and provides strong durability guarantees but introduces:

- Increased write latency due to replication.
- Temporary unavailability during leader election.

However, this trade-off is acceptable for financial workloads where correctness and durability are prioritized over minimal latency.

VII. DURABILITY VIA WRITE-AHEAD LOGGING

Durability is a fundamental requirement for financial transaction systems. Once a transaction is committed, it must survive process crashes, leader failures, and system restarts. The ledger achieves durability through a strictly enforced Write-Ahead Logging (WAL) mechanism.

A. Write-Ahead Logging Principle

The system follows the *log-before-apply* rule:

No state modification is applied to the ledger unless the corresponding log record has been durably persisted and replicated to a quorum.

This ordering ensures that recovery can always reconstruct committed state solely from log records.

B. WAL Record Structure

Each WAL entry contains at minimum:

- Log Identifier (logID)
- Transaction Identifier (txnID)
- Operation type (DEBIT, CREDIT, PREPARED, COMMITTED, ABORTED)
- Account identifiers
- Transfer amount
- Timestamp
- Transaction state marker

Entries are appended sequentially, forming an ordered log of shard-level operations.

C. WAL States and Transition Semantics

A transaction may appear in the WAL with different state markers:

- **UNCOMMITTED**: Entry appended but not yet quorum-confirmed.
- **PREPARED**: For cross-shard transactions during Phase 1 of 2PC.
- **COMMITTED**: Transaction is durable and ready for state application.
- **ABORTED**: Transaction will not be applied.

State transitions are monotonic. A transaction cannot revert from COMMITTED to any prior state.

D. Durable Persistence Semantics

For each log append:

- 1) The entry is written to the WAL buffer.
- 2) The WAL is synchronously flushed to stable storage (fsync).
- 3) The entry is replicated to followers.
- 4) Followers also persist via fsync before acknowledgment.

Only after quorum persistence is the commit index advanced.

This ensures that even if power loss occurs immediately after acknowledgment, the entry exists on disk on a majority of replicas.

Algorithm 5 Shard Crash Recovery

```
1: Read WAL sequentially from stable storage
2: for all log entries in order do
3:   if entry state is COMMITTED then
4:     Apply operation to ledger state
5:   else if entry state is PREPARED then
6:     Mark transaction for coordinator reconciliation
7:   else
8:     Ignore UNCOMMITTED or ABORTED entries
9:   end if
10: end for
11: Reconstruct in-memory ledger state
```

E. Crash Recovery Model

Upon shard restart, recovery proceeds as shown in Algorithm 5.

Recovery Semantics:

- Committed entries are replayed deterministically.
- Uncommitted entries are discarded.
- Prepared entries await final commit or abort decision from coordinator.

This guarantees that ledger state after restart is equivalent to the state immediately before crash.

F. Correctness Under Failure

1) *Crash Before WAL Flush:* If a crash occurs before fsync:

- The entry is absent from stable storage.
- The transaction is treated as non-existent.
- No state modification was applied.

2) *Crash After WAL Flush but Before Commit Index Advancement:* If the log entry was persisted but quorum was not achieved:

- The entry remains uncommitted.
- It is ignored during recovery.

3) *Crash After Quorum Commit:* If quorum acknowledgment occurred:

- The entry exists on a majority.
- The new leader contains the committed record.
- Recovery replay reconstructs committed state.

Thus, committed monetary transfers cannot be lost.

G. Durability in Cross-Shard Transactions

For cross-shard transactions:

- PREPARED state is durably logged before responding to the coordinator.
- COMMIT state is logged before applying updates.

Even if a crash occurs between PREPARE and COMMIT:

- The transaction remains in PREPARED state.
- The coordinator can safely reissue COMMIT or ABORT.
- Idempotent handling prevents double execution.

This ensures atomicity across shards despite failures.

H. Why Log-Before-Apply is Mandatory

If the system were to apply state changes before durable logging, the following failure scenario could occur:

- 1) Ledger state is modified.
- 2) Crash occurs before WAL flush.
- 3) Upon restart, the operation cannot be reconstructed.

This would violate conservation of funds and durability guarantees.

By enforcing WAL-before-apply ordering, the system ensures that:

Persist → Replicate → Commit → Apply

This strict ordering prevents state divergence and guarantees recoverability.

I. Performance Implications

While synchronous fsync operations introduce additional latency, they provide strong crash consistency guarantees. The durability model prioritizes financial correctness over minimal write latency.

Because WAL persistence occurs in sequential append-only fashion, disk overhead remains manageable even under high throughput.

J. Durability Guarantees

The WAL mechanism ensures:

- Committed transactions survive power loss.
- No partial ledger updates occur.
- Crash recovery reconstructs a consistent state.
- Cross-shard prepared states remain resolvable.

Combined with quorum-based replication and transaction coordination, write-ahead logging forms the foundation of system correctness under failure.

VIII. LOAD MONITORING AND PARTITION REBALANCING

To sustain high concurrency and prevent localized bottlenecks, the ledger incorporates a load monitoring and controlled partition rebalancing mechanism. The objective is to distribute workload evenly across shards while preserving transactional consistency and financial invariants.

A. Load Monitoring Architecture

Each shard periodically reports operational metrics to the Shard Load Monitor (SLM). These metrics include:

- Transaction throughput
- Request queue depth
- CPU and memory utilization
- Replication lag

Heartbeat messages serve two purposes:

- Detect node health and availability
- Provide real-time load visibility

The Load Monitor continuously evaluates shard metrics to detect sustained overload conditions.

Algorithm 6 Partition Migration Procedure

Require: Partition P , source shard S_1 , target shard S_2

- 1: Coordinator halts writes for P
 - 2: Flush pending WAL entries for P on S_1
 - 3: Transfer partition data from S_1 to S_2
 - 4: Verify data integrity on S_2
 - 5: Update Shard Map to map $P \rightarrow S_2$
 - 6: Resume writes for P
-

B. Overload Detection Policy

A shard is classified as overloaded if one or more of the following conditions persist beyond a defined threshold:

- Throughput significantly exceeds cluster average.
- Request latency exceeds predefined percentile bounds.
- Resource utilization approaches saturation.

Detection is conservative to avoid oscillatory migrations. Only sustained imbalance triggers rebalancing.

C. Partition-Based Rebalancing Model

Data is organized into logical partitions determined by deterministic account hashing. Rebalancing occurs at the partition level rather than at the individual account level.

Let P denote a logical partition currently mapped to shard S_1 .

When overload is detected:

- 1) The Load Monitor selects candidate partition P for migration.
- 2) The Load Monitor notifies the Transaction Coordinator to initiate migration.

D. Controlled Migration Procedure

To prevent inconsistent writes during migration, the system performs explicit partition halting and resumption.

Halt Phase: Before migration begins, the Coordinator temporarily halts new transactions targeting partition P . This ensures:

- No concurrent writes occur during transfer.
- WAL state is stable and fully replicated.

Data Transfer Phase: The source shard transfers:

- Current ledger state for partition P
- Associated WAL entries necessary for consistency

The target shard reconstructs partition state before activation.

Shard Map Update: After successful transfer, the Shard Map is updated atomically. Subsequent routing decisions by the Coordinator use the new mapping.

Resume Phase: Once the mapping is updated, halted transactions for partition P are resumed and routed to the new shard.

E. Consistency Guarantees During Migration

The migration protocol ensures:

- No transaction is partially applied across shards.
- No writes occur during partition transfer.
- Routing remains deterministic before and after migration.

Because the partition is halted prior to transfer and WAL entries are flushed, state divergence cannot occur.

F. Interaction with Replication

Partition migration occurs at the shard leader level. The destination shard establishes replication with its own follower replicas before accepting writes. Thus, quorum-based durability guarantees remain intact after migration.

G. Scalability Characteristics

Rebalancing operates at partition granularity, allowing:

- Incremental redistribution of load.
- Addition of new shards without global downtime.
- Gradual scaling proportional to workload growth.

Throughput increases as new shards are introduced and partitions are redistributed.

H. Limitations

While partition-level migration improves load distribution, the system cannot split a single highly contended account across multiple shards. If extreme contention is concentrated on one account, that account remains bound to a single partition.

This trade-off simplifies routing and preserves consistency but limits scalability for single-key hotspots.

I. Design Trade-offs

The rebalancing mechanism prioritizes consistency and simplicity:

- Explicit halting increases temporary latency for affected partitions.
- Conservative migration policies prevent instability.
- Partition-level granularity simplifies correctness reasoning.

These trade-offs ensure that load redistribution does not compromise financial safety or transactional guarantees.

IX. PRELIMINARY EVALUATION AND DESIGN ANALYSIS

At the current stage, the system architecture has been fully specified but not yet deployed in a production environment. Therefore, this section presents an analytical evaluation of performance characteristics, failure behavior, scalability projections, and design trade-offs based on the architectural model described earlier.

A. Analytical Latency Model

Transaction latency in the proposed system is dominated by synchronous log persistence and replication overhead.

1) *Single-Shard Latency*: A single-shard transaction requires:

- One WAL append
- One synchronous disk flush (fsync)
- One replication round to followers
- One quorum acknowledgment

Let:

$$T_{fsync} = \text{disk flush time}$$

$$T_{net} = \text{replication round-trip latency}$$

The approximate single-shard latency can be modeled as:

$$L_{single} \approx T_{fsync} + T_{net}$$

Because WAL writes are sequential and replication is parallelized to followers, the latency remains bounded primarily by storage persistence and network delay.

2) *Cross-Shard Latency*: Cross-shard transactions incur additional coordination overhead due to two-phase commit (2PC). Each participating shard performs:

- PREPARE WAL append + replication
- COMMIT WAL append + replication

Additionally, the coordinator introduces at least one extra messaging round.

The approximate latency becomes:

$$L_{cross} \approx 2(T_{fsync} + T_{net}) + T_{coord}$$

where T_{coord} represents coordinator communication delay.

Thus, cross-shard transactions are expected to exhibit roughly twice the persistence cost of single-shard transactions.

B. Throughput Scalability Projection

Because account data is deterministically partitioned, each shard operates independently for single-shard transactions.

If each shard sustains a minimum throughput of 100 transactions per second (TPS), the projected aggregate throughput becomes:

$$Throughput_{total} = N_{shards} \times 100$$

Number of Shards	Projected Throughput (TPS)
1	100
4	400
8	800
16	1600

This projection assumes a majority of transactions are single-shard. Workloads dominated by cross-shard transfers would experience additional coordination overhead, reducing effective throughput.

C. Failure Scenario Validation

The architecture was evaluated against representative failure cases to validate correctness guarantees.

1) *Case 1: Crash Before WAL Flush*: If a crash occurs before the WAL entry is flushed to disk:

- No durable record exists.
- No state modification has been applied.
- The transaction is safely discarded.

Financial invariants remain intact.

2) *Case 2: Crash After WAL Flush but Before Quorum*: If the WAL entry is persisted locally but quorum acknowledgment is not achieved:

- The entry remains uncommitted.
- During recovery, it is ignored.
- No ledger update is applied.

No partial state becomes externally visible.

3) *Case 3: Crash After Quorum Commit*: If quorum acknowledgment occurs before crash:

- The entry exists on a majority of replicas.
- The promoted leader contains the committed record.
- Recovery replay reconstructs committed state.

Committed monetary transfers remain durable.

D. Consistency Under Cross-Shard Failures

During cross-shard transactions:

- PREPARED entries are durably logged before acknowledgment.
- COMMIT decisions are applied only after all shards confirm PREPARED.
- ABORT decisions revert prepared state consistently.

Even if a coordinator fails between phases, replayed messages are handled idempotently, preventing duplicate execution.

Thus, atomicity across shards is preserved despite partial failures.

E. Rebalancing Impact Analysis

Partition migration introduces temporary write suspension for affected partitions. While this increases latency for that subset of accounts, it prevents inconsistent routing and partial updates.

The system cannot split a single highly contended account across multiple shards. Therefore, extreme single-key hotspots remain a scalability limitation.

F. Trade-Off Analysis

The design prioritizes correctness and durability over minimal latency.

- Synchronous fsync increases write latency but guarantees crash consistency.
- Majority replication increases fault tolerance but adds network overhead.
- Two-phase commit ensures atomicity but introduces coordination delay.
- Partition-level rebalancing simplifies reasoning but limits hotspot mitigation.

These trade-offs are deliberate, reflecting the requirements of financial systems where correctness outweighs raw throughput.

G. Current Limitations and Future Evaluation

The present evaluation is analytical. Empirical benchmarking is planned to measure:

- p99 latency under varying shard counts.
- Throughput scaling under concurrent clients.
- Recovery time after primary failure.
- Overhead of synchronous fsync on SSD vs HDD.

Future work will validate projected performance metrics and refine migration heuristics based on observed workload characteristics.

X. CONCLUSION

This paper presented the design of a High-Concurrency Distributed Ledger Service built upon deterministic sharding, quorum-based replication, write-ahead logging, and two-phase commit coordination. The architecture separates transaction ingestion, coordination, persistence, and operational monitoring into clearly defined layers, enabling horizontal scalability while preserving strict financial correctness guarantees.

The proposed transaction processing model ensures atomicity for both single-shard and cross-shard transfers. Majority-based replication and log-before-apply semantics provide durability under crash and failover scenarios. Partition-level load monitoring and controlled migration allow dynamic redistribution of workload without compromising consistency.

An analytical evaluation demonstrated the expected latency characteristics of single-shard and cross-shard transactions and projected linear throughput scaling with the addition of shards. Failure scenario validation confirmed that the design preserves ledger invariants under primary crashes, partial replication, and coordinator failures.

The system deliberately prioritizes correctness, recoverability, and transparency over aggressive performance optimization. This design choice reflects the requirements of financial transaction systems, where safety and durability outweigh minimal write latency.

Future work will focus on empirical benchmarking, optimization of replication latency, refinement of migration heuristics, and evaluation under high-contention workloads. Additional enhancements may include non-blocking consensus protocols and improved hotspot mitigation strategies.

The presented architecture provides a rigorous foundation for understanding distributed transaction systems and demonstrates how strong consistency guarantees can be maintained in a scalable, sharded environment.

AI USAGE DECLARATION

AI-assisted tools were used for language refinement, structural organization, and formatting guidance in preparing this report. All architectural decisions, system design, algorithms, analytical evaluation, and technical reasoning were independently developed by the authors. The use of AI tools did not replace conceptual understanding or original system design work.

REFERENCES

- [1] J. C. Corbett et al., “Spanner: Google’s Globally-Distributed Database,” OSDI, 2012.
- [2] Cockroach Labs, “CockroachDB Architecture,” 2023.
- [3] A. Kreps et al., “Kafka: A Distributed Messaging System,” 2011.
- [4] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.
- [5] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” CACM, 1978.